

Musterlösung — Übungsklausur 10

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. CNNs vs. MLPs für Bilder?

✓ (b) CNNs: Parameter-Sharing + Ausnutzung von Nachbarschaftskorrelationen → statistisch effizienter

- (a) Mehr Parameter
- (c) Können keine Bilder klassifizieren
- (d) Mehr Trainingsdaten

2. Log-Loss Aussage?

✓ (b) Penalisiert auch korrekt klassifizierte Punkte nahe der Grenze (Margin-Effekt)

- (a) Nur Fehlklassifikationen
- (c) Identisch mit Squared Loss
- (d) Gradient null

3. Skip Connection ResNet?

✓ (b) $x_{l+1} = x_l + \text{NN}_l(x_l; \theta_l)$

- (a) Nur NN ohne Addition
- (c) Multiplikation
- (d) Addition vorheriger zwei Layer

4. Activation Maximization vs. Attribution?

✓ (b) Activation Max: global (was hat Modell gelernt). Attribution: lokal (einzelne Vorhersage erklären)

- (a) Umgekehrt
- (c) Äquivalent
- (d) Heatmaps ohne Input

5. Spurious Correlations?

✓ (b) Task-irrelevante Variablen zufällig mit relevanten korreliert in P — Modell kann darauf basieren

- (a) Immer in Labels
- (c) Nur kleine Datensätze
- (d) Aus Testdaten lesbar

6. Conv2d(3,20,5) Gewichte ohne Bias?

✓ (b) 1500 (= $20 \times 3 \times 5 \times 5$)

- (a) 300
- (c) 6000
- (d) 75

7. Activation Maximization Ziel?

✓ (a) $x^* = \text{argmax}_x f(x)$ — maximiert spezifischen Ausgabe-Neuron

- (b) Loss maximieren
- (c) Attribution berechnen
- (d) Adversarielle Robustheit

8. Cell State wenn alle Gates geschlossen?

✓ (b) c bleibt konstant — kein Schreiben, Lesen oder Löschen

- (a) Auf null gesetzt

- (c) Mit Input aktualisiert
- (d) Exponentiell abgeschwächt

2 Theorie — Klassifikationslosses im Vergleich

(a) Vergleichstabelle der Losses

0/1-Loss: $\ell = 1_{\{y \cdot t \leq 0\}}$. Nicht via GD optimierbar (Gradient=0). Korrekte Punkte: Penalty=0. Kein Margin.

Perceptron Loss: $\ell = \max(0, -y \cdot t)$. Via GD optimierbar (für Fehlklassifikationen). Korrekte Punkte: Penalty=0. Kein Margin.

Log-Loss: $\ell = \log(1 + \exp(-y \cdot t))$. Via GD optimierbar. Korrekte Punkte: Penalty>0 (wenn nahe Grenze). Impliziter Margin.

(b) Log-Loss als Regularisierer

Log-Loss bleibt für korrekte, aber unsichere Vorhersagen ($y \cdot t$ klein, nahe 0) signifikant >0. Das Netz wird bestraft, auch wenn Klassifikation korrekt. Um Penalty zu minimieren, muss $y \cdot t$ groß werden → Entscheidungsgrenze wird von Trainingsdaten weggeschoben → impliziter Margin.

(c) Cross-Entropy & MLE

MLE: Maximiere $\sum_i \log p(t_i | x_i, \theta)$. Unter Multinomialverteilung $p_i = \text{softmax}(y_i)$:

$$-\log p(t|x) = -\sum_c q_c \cdot \log p_c = H(q, p) = \text{Cross-Entropy}$$

Minimierung des Cross-Entropy Loss $\equiv$ Maximum-Likelihood-Schätzung unter Multinomialverteilung. Natural für diskrete Klassifikation.

3 Theorie — CNN-Anwendungen

(a) YOLO Objektdetektion

YOLO (You Only Look Once): Teilt Bild in $S \times S$ -Grid. Jede Zelle sagt B Bounding Boxes vorher: (x, y, w, h, Konfidenz) + C Klassenwahrscheinlichkeiten. Loss: Kombination aus Lokalisierungs-, Konfidenz- und Klassifikationsverlust.

(b) WaveNet & Dilated Convolutions

WaveNet: Autoregressive Wellenformgenerierung. Dilated Causal Convolutions: Kernel-Punkte sind durch 'Lücken' (Dilation Rate d) getrennt → exponentiell wachsendes Receptive Field ohne mehr Parameter. Stack von Schichten mit $d=1,2,4,8,16,\dots$ → großer Kontext bei wenig Rechenaufwand.

(c) Kausalität bei Faltungen

'Kausal' bedeutet: Output zum Zeitpunkt t hängt nur von Inputs bis t ab (keine Zukunftsinformation).

Formal: Kernel $w(\tau) = 0$ für $\tau < 0$. Essentiell für Generierung: beim Erzeugen von Sample t ist Sample t+1 noch nicht bekannt → autoregressive Generierung möglich.

4 Theorie — RNN BPTT (Komplette Herleitung)

(a) Berechnungsgraph

Knoten: $x_t, x_{t-1}, w, h_t=0$. Berechnung: $h_t = \tanh(x_t w + h_{t-1})$, $y_t = h_t$, $E = (y_t - t)^2$.

(b) $\partial E / \partial y$ und $\partial E / \partial h_t$

$$\partial E / \partial y = 2(y - t)$$

$$\partial E / \partial h = \partial E / \partial y \cdot \partial y / \partial h = 2(y-t) \cdot 1 = 2(y-t) \quad [\text{da } y=h]$$

(c) Kettenregel $\partial E / \partial w$

w erscheint in h (via $x \cdot w$) und in h (via $x \cdot w$). BPTT: beide Pfade summieren.

$$\partial E / \partial w = \partial E / \partial h \cdot \partial h / \partial w + \partial E / \partial h \cdot \partial h / \partial h \cdot \partial h / \partial w$$

$$\partial h / \partial w = (1-h^2) \cdot x$$

$$\partial h / \partial h = (1-h^2)$$

$$\partial h / \partial w = (1-h^2) \cdot x$$

(d) Explizite Berechnung

$$\partial E / \partial w = 2(y-t) \cdot [(1-h^2) \cdot x + (1-h^2) \cdot (1-h^2) \cdot x]$$

→ **Erster Term: direkter Pfad durch h . Zweiter Term: BPTT-Pfad durch $h \rightarrow h$.**

5 Programmierung — Multi-Task Learning

(a) Multi-Task Modell

```
class MultiTaskNet(nn.Module):
    def __init__(self, in_dim=64):
        super().__init__()
        self.shared = nn.Sequential(
            nn.Linear(in_dim, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU())
        self.head_reg = nn.Linear(64, 1) # Regression
        self.head_cls = nn.Linear(64, 1) # Klassifikation
        def forward(self, x):
            feat = self.shared(x)
            return self.head_reg(feat), self.head_cls(feat)
```

(b) Forward Pass & kombinierter Loss

```
y_reg, y_cls = model(x)
loss_reg = nn.MSELoss()(y_reg.squeeze(), t_reg)
loss_cls = nn.BCEWithLogitsLoss()(y_cls.squeeze(), t_cls)
loss = loss_reg + lambda_cls * loss_cls
```

Gewichtung λ : Via Holdout Validation tunen. Alternativ: Uncertainty-basiertes Gewichten nach Kendall et al. (Tasks mit hoher aleatoric Uncertainty werden geringer gewichtet, da Modell ohnehin unsicher ist).